AI SMART AGRICULTURE

Full Updated Code + Complete Project Explanation

1. Project Introduction

AI Smart Agriculture is a web-based project that uses Artificial Intelligence, Machine Learning and agricultural environmental data to support smart irrigation decisions. The system works with soil moisture, temperature, humidity, rainfall and crop growth stage.

2. Project Objective

The main objective is to reduce unnecessary water usage and help farmers make data-based irrigation decisions. The system predicts whether irrigation is required and provides a recommended water quantity for demonstration purposes.

3. Inputs Used

- Soil moisture
- Temperature
- Humidity
- Rainfall
- Crop growth stage

4. Machine Learning Model

The project uses a Random Forest Classifier. Demonstration sensor records are generated and divided into training and testing data. The trained model predicts whether irrigation is needed. Model confidence is also returned to the dashboard.

5. Irrigation Decision

The final irrigation decision combines the Machine Learning prediction with a rule-based irrigation plan. Rainfall and adequate soil moisture can prevent unnecessary irrigation. The project also estimates recommended litres using moisture deficit, temperature, humidity, rainfall and crop stage.

6. Crop Condition

The system calculates a simple crop score and classifies the condition as Healthy, Moderate or Critical. This classification is based on the demonstration environmental values used by the project.

7. Dashboard Features

The dashboard displays sensor values, accepts field inputs, shows irrigation status, crop condition, crop score, model confidence, recommended water quantity and the reason for the decision. Recent sensor readings are displayed in a table and automatically refreshed.

8. Technologies Used

Python, Flask, Pandas, Scikit-learn, Random Forest, Joblib, HTML, CSS and JavaScript.

9. Project Structure

```
AI_Smart_Agriculture/
|-- app.py
```

```
|-- requirements.txt
|-- templates/
|   `-- index.html
`-- static/
    |-- style.css
    `-- script.js
```

10. Complete app.py Code

```
from flask import Flask, render_template, jsonify, request
import os
import random
import pandas as pd
import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

app = Flask(__name__)

DATA_FILE = "sensor_data.csv"
MODEL_FILE = "irrigation_model.pkl"
FEATURES = [
    "soil_moisture",
    "temperature",
    "humidity",
    "rainfall",
    "crop_stage"
]

def generate_seed_data(n=400):
    rows = []

    for _ in range(n):
        soil = random.randint(10, 80)
        temp = random.randint(18, 42)
        humidity = random.randint(20, 95)
        rainfall = random.randint(0, 35)
        stage = random.randint(1, 5)

        irrigation = int(
            soil < 35 and
            temp > 28 and
            humidity < 55 and
            rainfall < 15
        )

        if rainfall >= 15:
            irrigation = 0

        if soil < 25 or humidity < 35:
            condition = "Critical"
        elif soil < 40 or humidity < 55:
            condition = "Moderate"
        else:
            condition = "Healthy"

        rows.append({
            "soil_moisture": soil,
            "temperature": temp,
            "humidity": humidity,
            "rainfall": rainfall,
            "crop_stage": stage,
            "needs_irrigation": irrigation,
            "crop_condition": condition
        })

    pd.DataFrame(rows).to_csv(DATA_FILE, index=False)

def train_model():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE)

    X = df[FEATURES]
    y = df["needs_irrigation"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    model = RandomForestClassifier(
        n_estimators=200,
        random_state=42
    )

    model.fit(X_train, y_train)

    accuracy = accuracy_score(
        y_test,
        model.predict(X_test)
    )

    print("Model Accuracy:", round(accuracy * 100, 2), "%")

    joblib.dump(model, MODEL_FILE)
    return model

def load_model():
    if os.path.exists(MODEL_FILE):
        return joblib.load(MODEL_FILE)

    return train_model()

def get_crop_condition(data):
    score = 0

    if data["soil_moisture"] < 25:
        score -= 3
    elif data["soil_moisture"] < 40:
        score -= 1
    else:
```

```
        score += 2

    if data["temperature"] > 35:
        score -= 3
    elif data["temperature"] > 30:
        score -= 1
    else:
        score += 1

    if data["humidity"] < 35:
        score -= 3
    elif data["humidity"] < 55:
        score -= 1
    else:
        score += 2

    if data["rainfall"] > 15:
        score += 2
    elif data["rainfall"] > 5:
        score += 1
    else:
        score -= 1

    if score >= 3:
        return "Healthy", score
    elif score >= 0:
        return "Moderate", score
    else:
        return "Critical", score

def calculate_irrigation_plan(data):
    if data["rainfall"] >= 12:
        return (
            False,
            0.0,
            "Rainfall is sufficient; irrigation is skipped."
        )

    if (
        data["soil_moisture"] > 55 and
        data["humidity"] > 65
    ):
        return (
            False,
            0.0,
            "Soil moisture and humidity are adequate."
        )

    crop_factor = {
        1: 0.8,
        2: 1.0,
        3: 1.2,
        4: 1.1,
        5: 0.9
    }.get(data["crop_stage"], 1.0)

    moisture_deficit = max(
        0,
        50 - data["soil_moisture"]
    )

    temperature_load = max(
        0,
        data["temperature"] - 25
    ) * 0.7

    humidity_load = max(
        0,
        65 - data["humidity"]
    ) * 0.25

    litres = (
        moisture_deficit * 1.6 +
        temperature_load +
        humidity_load
    ) * crop_factor

    if data["rainfall"] > 0:
        litres *= 0.6

    litres = min(max(litres, 0), 65)
    litres = round(litres, 1)

    return (
        litres > 2,
        litres,
        "Targeted irrigation is recommended."
    )

model = load_model()

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/health")
def health():
    return jsonify({"status": "ok"})

@app.route("/api/sensors")
def sensors():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE)

    return jsonify(
        df.tail(10).to_dict(orient="records")
    )

@app.route("/api/predict", methods=["POST"])
```

```
def predict():
    data = request.get_json() or {}

    sample = {
        "soil_moisture": float(
            data.get("soil_moisture", 30)
        ),
        "temperature": float(
            data.get("temperature", 28)
        ),
        "humidity": float(
            data.get("humidity", 50)
        ),
        "rainfall": float(
            data.get("rainfall", 5)
        ),
        "crop_stage": int(
            data.get("crop_stage", 2)
        )
    }

    frame = pd.DataFrame([sample])

    prediction = bool(
        model.predict(frame)[0]
    )

    confidence = float(
        max(model.predict_proba(frame)[0])
    )

    plan_need, litres, reason = (
        calculate_irrigation_plan(sample)
    )

    final_need = prediction and plan_need

    if not final_need:
        litres = 0.0
        reason = (
            "Irrigation avoided to save water "
            "and reduce unnecessary usage."
        )

    condition, score = get_crop_condition(sample)

    return jsonify({
        "needs_irrigation": final_need,
        "recommended_liters": litres,
        "confidence": round(confidence, 2),
        "crop_condition": condition,
        "crop_score": score,
        "reason": reason,
        "input": sample
    })

if __name__ == "__main__":
    app.run(
        debug=True,
        host="0.0.0.0",
        port=5000
    )
```

11. Complete index.html Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>AI Smart Agriculture</title>

    <link rel="stylesheet"
          href="{{ url_for('static',
          filename='style.css') }}">
</head>

<body>

<div class="container">

    <h1>AI Smart Agriculture Dashboard</h1>

    <div class="cards">

        <div class="card">
            <h3>Soil Moisture</h3>
            <p id="soilValue">--</p>
        </div>

        <div class="card">
            <h3>Temperature</h3>
            <p id="tempValue">--</p>
        </div>

        <div class="card">
            <h3>Humidity</h3>
            <p id="humidityValue">--</p>
        </div>

        <div class="card">
            <h3>Rainfall</h3>
            <p id="rainValue">--</p>
        </div>

    </div>

    <div class="panel">

        <h2>AI Irrigation Prediction</h2>

        <form id="predictForm">

            <label>Soil Moisture (%)</label>
            <input id="soil_moisture"
                   type="number"
                   value="32">

            <label>Temperature (°C)</label>
            <input id="temperature"
                   type="number"
                   value="30">

            <label>Humidity (%)</label>
            <input id="humidity"
                   type="number"
                   value="45">

            <label>Rainfall (mm)</label>
            <input id="rainfall"
                   type="number"
                   value="5">

            <label>Crop Stage</label>

            <select id="crop_stage">
                <option value="1">Early Growth</option>
                <option value="2" selected>
                    Vegetative
                </option>
                <option value="3">Flowering</option>
                <option value="4">Fruiting</option>
                <option value="5">Harvest</option>
            </select>

            <button type="submit">
                Analyze
            </button>

        </form>

        <div id="result">
            Enter field values and click Analyze.
        </div>

    </div>

    <div class="panel">

        <h2>Recent Sensor Data</h2>

        <table>

            <thead>
                <tr>
                    <th>Soil</th>
                    <th>Temperature</th>
                    <th>Humidity</th>
                    <th>Rainfall</th>
                    <th>Stage</th>
```

```
                    <th>Condition</th>
                    <th>Irrigation</th>
                </tr>
            </thead>

            <tbody id="sensorTableBody"></tbody>

        </table>

    </div>

</div>

<script src="{{ url_for('static',
        filename='script.js') }}"></script>

</body>
</html>
```

12. Complete style.css Code

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background: #f3f7f3;
    color: #1d2b1d;
}

.container {
    max-width: 1050px;
    margin: auto;
    padding: 25px;
}

h1 {
    text-align: center;
    color: #236b2a;
}

.cards {
    display: grid;
    grid-template-columns:
        repeat(auto-fit, minmax(180px, 1fr));
    gap: 15px;
}

.card,
.panel {
    background: white;
    padding: 18px;
    border-radius: 12px;
    box-shadow: 0 2px 8px #0001;
}

.card p {
    font-size: 25px;
    font-weight: bold;
}

.panel {
    margin-top: 20px;
}

label {
    display: block;
    margin-top: 12px;
    margin-bottom: 6px;
    font-weight: bold;
}

input,
select {
    width: 100%;
    padding: 10px;
    box-sizing: border-box;
    border: 1px solid #ccd5cc;
    border-radius: 8px;
}

button {
    margin-top: 15px;
    padding: 12px 20px;
    background: #2e7d32;
    color: white;
    border: 0;
    border-radius: 8px;
    cursor: pointer;
}

#result {
    margin-top: 15px;
    padding: 16px;
    border-radius: 10px;
    background: #edf7ed;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th,
td {
    padding: 9px;
    border-bottom: 1px solid #ddd;
    text-align: center;
}

th {
    background: #dcefdc;
}
```

13. Complete script.js Code

```
async function loadSensors() {
    const response = await fetch("/api/sensors");
    const rows = await response.json();

    const body =
        document.getElementById("sensorTableBody");

    body.innerHTML = "";

    rows.forEach(row => {

        const tr = document.createElement("tr");

        tr.innerHTML = `
            <td>${row.soil_moisture}%</td>
            <td>${row.temperature}°C</td>
            <td>${row.humidity}%</td>
            <td>${row.rainfall} mm</td>
            <td>${row.crop_stage}</td>
            <td>${row.crop_condition}</td>
            <td>
                ${row.needs_irrigation ? "Yes" : "No"}
            </td>
        `;

        body.appendChild(tr);
    });

    if (rows.length > 0) {

        const latest = rows[rows.length - 1];

        document.getElementById("soilValue")
            .textContent =
            latest.soil_moisture + "%";

        document.getElementById("tempValue")
            .textContent =
            latest.temperature + "°C";

        document.getElementById("humidityValue")
            .textContent =
            latest.humidity + "%";

        document.getElementById("rainValue")
            .textContent =
            latest.rainfall + " mm";
    }
}

document.getElementById("predictForm")
.addEventListener("submit", async function(event) {

    event.preventDefault();

    const payload = {
        soil_moisture:
            Number(document.getElementById(
                "soil_moisture").value),

        temperature:
            Number(document.getElementById(
                "temperature").value),

        humidity:
            Number(document.getElementById(
                "humidity").value),

        rainfall:
            Number(document.getElementById(
                "rainfall").value),

        crop_stage:
            Number(document.getElementById(
                "crop_stage").value)
    };

    const response = await fetch(
        "/api/predict",
        {
            method: "POST",
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(payload)
        }
    );

    const data = await response.json();

    document.getElementById("result").innerHTML = `
        <b>
            ${data.needs_irrigation
                ? "IRRIGATION REQUIRED"
                : "NO IRRIGATION NEEDED"}
        </b>

        <p>
            Crop Condition:
            ${data.crop_condition}
        </p>

        <p>
            Crop Score:
            ${data.crop_score}
        </p>
```

```
        <p>
            Confidence:
            ${(data.confidence * 100).toFixed(1)}%
        </p>

        <p>
            Recommended Water:
            ${data.recommended_liters} litres
        </p>

        <p>
            Reason:
            ${data.reason}
        </p>
    `;
});

loadSensors();

setInterval(loadSensors, 3000);
```

14. requirements.txt

```
Flask
pandas
scikit-learn
joblib
```

15. How the Project Works

Step 1: The Flask application starts.
Step 2: Demonstration sensor data is generated if required.
Step 3: A Random Forest model is trained and saved.
Step 4: The dashboard loads recent sensor readings.
Step 5: The user enters current field conditions.
Step 6: The backend sends the values to the ML model.
Step 7: Irrigation rules also check rainfall and moisture.
Step 8: The system calculates crop condition and water quantity.
Step 9: The result is displayed on the dashboard.

16. How to Run

Create the folders exactly as shown in the project structure. Install the packages using the requirements file. Then run **python app.py**. Open the Flask local address in a browser. The dashboard can then be used to test different agricultural conditions.

17. Expected Output

The user can see sensor readings and submit field conditions. The dashboard returns whether irrigation is required, crop condition, crop score, prediction confidence, recommended water quantity and the reason for the decision.

18. Benefits

• Supports data-based irrigation decisions
• Helps reduce unnecessary watering
• Combines ML prediction and environmental conditions
• Provides a simple dashboard for monitoring
• Demonstrates AI use in agriculture
• Can be extended with real sensors and real datasets

19. Future Enhancement

The project can be improved by connecting real soil-moisture and weather sensors, using a larger real-world agricultural dataset, adding plant disease detection, adding weather APIs, and storing historical farm data for long-term analysis.

20. Conclusion

AI Smart Agriculture demonstrates how Artificial Intelligence and Machine Learning can be applied to agriculture. By combining sensor-related environmental values with an ML prediction and water-saving logic, the system provides a practical educational example of smart irrigation and crop monitoring.

Note: The water recommendation in this student project is a demonstration calculation. Real agricultural deployment requires calibrated sensors, crop-specific data and field validation.